

MedOrbit – Smart Healthcare Platform

Software Requirements Specification (SRS)

Document Version: 2.2
Date: 2026-07-02
Status: Draft for Review
Project Type: Graduation Project – Advanced Software Engineering
Authors: Graduation Project Team

Table of Contents

- 1. Introduction
 - 2. Overall Description
 - 3. System Features & Functional Requirements
 - 4. AI Modules
 - 5. External Interface Requirements
 - 6. Non-Functional Requirements
 - 7. Architecture Overview
 - 8. Technology Stack & Justification
 - 9. Project Phases & Deliverables
 - 10. Acceptance Criteria
-

1. Introduction

1.1 Purpose

This document specifies the complete software requirements for **MedOrbit**, a unified smart healthcare platform designed to consolidate medical services into a single, intelligent system. The platform targets patients, doctors, and administrators, providing end-to-end healthcare management through web and mobile interfaces. This SRS serves as the authoritative reference for development, testing, and validation.

1.2 Intended Audience

- **Development Team:** Backend, frontend, mobile, and AI engineers
- **Project Supervisor:** Academic advisor and defense committee
- **Project Manager:** Team lead coordinating sprints and deliverables
- **QA Team:** Responsible for test planning and validation

1.3 Product Scope

MedOrbit addresses the fragmentation of healthcare services by unifying patient management, appointment scheduling, telemedicine, electronic medical records, e-prescription, AI-powered assistance, and analytics into one cohesive ecosystem. The platform supports bilingual operation (Arabic RTL and English LTR) and integrates modern AI capabilities to enhance clinical decision-making and patient experience.

1.4 Definitions & Acronyms

Term	Definition
EMR	Electronic Medical Record
RAG	Retrieval-Augmented Generation
LLM	Large Language Model
RBAC	Role-Based Access Control
JWT	JSON Web Token
WebRTC	Web Real-Time Communication
SOAP	Subjective, Objective, Assessment, Plan (medical note format)
Triage	Preliminary clinical assessment of symptom severity
FCM	Firebase Cloud Messaging
DLQ	Dead Letter Queue

2. Overall Description

2.1 Product Perspective

MedOrbit is a greenfield system designed as a distributed microservices architecture. It operates as a standalone platform but integrates with external services such as Google Maps, Firebase Cloud Messaging, email gateways, and AI model providers. The system exposes a unified REST API consumed by a web client (HTML/CSS/JS) and a mobile client (Flutter).

2.2 User Classes & Characteristics

User Class	Description	Primary Needs
Patients	End-users seeking medical services	Book appointments, consult doctors, manage health records, receive medication reminders
Doctors	Licensed medical professionals	Manage availability, conduct consultations, write prescriptions, access patient records
Administrators	System operators and hospital IT staff	Monitor analytics, verify doctors, manage users, configure system parameters
Donors	External contributors (optional phase)	Fund treatments, track sponsorship transparency

2.3 Operating Environment

- **Web Client:** Modern browsers (Chrome, Firefox, Safari, Edge) supporting ES6+, WebSocket, and WebRTC
- **Mobile Client:** Android 8.0+ and iOS 14+ via Flutter cross-platform compilation
- **Backend Services:** Linux-based containers (Docker) with Node.js 20 LTS and Python 3.11 run-times
- **Database:** PostgreSQL 15+ per service; Redis for caching; FAISS/ChromaDB for vector storage
- **Network:** HTTPS/TLS 1.2+ for all communications; WebSocket for real-time features

2.4 Design & Implementation Constraints

- **MySQL/PostgreSQL Mandatory:** The course requires a relational database; PostgreSQL is selected for its advanced JSONB support and vector extension compatibility.
- **Backend Flexibility:** Node.js (Express/Fastify) is chosen for core services due to its non-blocking I/O model, which aligns with real-time healthcare communication needs. Python (FastAPI) is mandated for AI services due to its rich ML ecosystem.
- **Microservices Architecture:** Required to demonstrate scalability, independent deployment, and service isolation.
- **Event-Driven Communication:** Apache Kafka is required for asynchronous workflows (e.g., appointment notifications, prescription alerts).
- **Bilingual Support:** Full Arabic (RTL) and English (LTR) support is mandatory for the Palestinian and regional market.
- **Security Compliance:** All medical data must be encrypted at rest and in transit; audit logging is mandatory for every medical record access.

2.5 Assumptions & Dependencies

- Doctors will provide valid license numbers for verification by administrators.
- Patients will have access to smartphones or web browsers with stable internet for video consultations.
- External AI APIs (OpenAI) or local models (Llama via Ollama) will be available during development and deployment.
- Google Maps API keys will be procured for location-based services.
- Firebase project will be configured for push notifications.

3. System Features & Functional Requirements

3.1 Authentication & Authorization

The system shall provide secure, role-based access control for all user classes.

- **FR-1.1:** The system shall allow users to register as Patient, Doctor, or Admin, capturing email, password, phone, full name, role, and preferred language.
- **FR-1.2:** The system shall validate email addresses via OTP before account activation.
- **FR-1.3:** The system shall support secure login with JWT access tokens (15-minute expiry) and refresh tokens (7-day rotating expiry).
- **FR-1.4:** The system shall enforce RBAC with three roles: `admin`, `doctor`, `patient`.
- **FR-1.5:** The system shall lock accounts after 5 failed login attempts for 15 minutes.
- **FR-1.6:** The system shall enforce password complexity (minimum 8 characters, uppercase, lowercase, digit, special character) and hash passwords using `bcrypt` (cost factor 12).
- **FR-1.7:** The system shall support password reset via secure email token and logout via refresh token invalidation.

3.2 User Management

- **FR-2.1:** Users shall be able to create and update profiles (name, photo, phone, address, preferred language).

- **FR-2.2:** Patients shall maintain medical information: blood type, allergies, chronic diseases, and current medications.
- **FR-2.3:** Doctors shall maintain professional profiles: specialty, license number, bio, years of experience, and consultation fee.
- **FR-2.4:** Admins shall be able to list, search, suspend, or delete users; verify doctor credentials before public visibility.
- **FR-2.5:** Users shall be able to soft-delete their accounts, preserving medical records for regulatory compliance.

3.3 Appointment Management

- **FR-3.1:** Patients shall be able to book appointments with doctors at available time slots, with three types: `in_person`, `voice_call`, `video_call`.
- **FR-3.2:** Patients shall be able to cancel or reschedule appointments with a reason.
- **FR-3.3:** Doctors shall define and manage weekly availability (day, start time, end time, slot duration).
- **FR-3.4:** The system shall support a waiting list when slots are fully booked.
- **FR-3.5:** The system shall provide smart appointment suggestions based on the patient's triage classification and doctor availability.
- **FR-3.6:** The system shall mark patients as `no_show` if they do not join within 15 minutes of the scheduled time.
- **FR-3.7:** Patients shall be able to rate and review doctors after completed appointments.
- **FR-3.8:** The system shall send appointment reminders 24 hours and 1 hour before the scheduled time.

3.4 Telemedicine

- **FR-4.1:** The system shall support secure video consultations between doctor and patient using WebRTC.
- **FR-4.2:** The system shall support real-time chat during consultations via WebSocket.
- **FR-4.3:** The system shall support voice calls as an alternative to video.
- **FR-4.4:** The system shall allow file sharing (images, PDFs) within the chat interface.
- **FR-4.5:** Doctors shall be able to take consultation notes during or after the call.
- **FR-4.6:** The system shall record consultation metadata: start time, end time, duration, and participants.
- **FR-4.7:** Both participants shall be able to review chat history after the call ends.

3.5 Electronic Medical Records (EMR)

- **FR-5.1:** Doctors shall create and update medical records per consultation in SOAP format (Subjective, Objective, Assessment, Plan).
- **FR-5.2:** The system shall maintain a patient's complete medical history, including past illnesses, surgeries, and family history.
- **FR-5.3:** The system shall allow doctors to upload laboratory reports and medical documents (text, PDF, images).
- **FR-5.4:** Patients shall be able to upload medical documents from external clinics for their personal archive.
- **FR-5.5:** Medical records shall be version-controlled with a full audit trail (who, when, what action).

- **FR-5.6:** The system shall automatically invoke AI Report Summarization when a long report is uploaded.
- **FR-5.7:** All medical record access shall be audit-logged and retained for a minimum of one year.

3.6 E-Prescription Management

- **FR-6.1:** Doctors shall be able to create electronic prescriptions containing medications (name, dosage, frequency, duration, instructions).
- **FR-6.2:** The system shall automatically call the Drug Interaction Service before saving a prescription.
- **FR-6.3:** If a **major** or **contraindicated** interaction is detected, the system shall require explicit doctor confirmation.
- **FR-6.4:** The system shall generate a PDF version of the prescription for download.
- **FR-6.5:** Patients shall be able to view their active and past medication history.
- **FR-6.6:** The system shall send medication reminders via the Notification Service based on the prescription schedule.
- **FR-6.7:** Doctors shall be able to revoke prescriptions with a documented reason.

3.7 Notification System

- **FR-7.1:** The system shall send appointment reminders (24h and 1h before).
- **FR-7.2:** The system shall send medication reminders based on prescription schedules.
- **FR-7.3:** The system shall support push notifications (FCM for mobile), email (SMTP), and in-app real-time notifications (WebSocket).
- **FR-7.4:** Users shall be able to mark notifications as read and configure preferences per category.

3.8 Location & GPS Services

- **FR-8.1:** Users shall be able to share their current location with permission.
- **FR-8.2:** The system shall locate nearby hospitals, clinics, and pharmacies within a configurable radius.
- **FR-8.3:** The system shall integrate with Google Maps for visualization and turn-by-turn navigation.
- **FR-8.4:** Facility data shall be cached in Redis for fast retrieval.

3.9 Multi-Language Support

- **FR-9.1:** The system shall provide a full Arabic interface (RTL layout) and English interface (LTR layout).
- **FR-9.2:** The system shall support dynamic language switching without page reload.
- **FR-9.3:** All user-facing strings shall be externalized in JSON dictionary files (web) and ARB files (mobile).
- **FR-9.4:** The AI Chatbot and Triage questions shall be displayed in the user's selected language.

4. AI Modules

The platform integrates five AI capabilities delivered through four Python microservices. These modules differentiate MedOrbit from conventional healthcare management systems by providing intelligent assistance, risk assessment, and data-driven insights.

4.1 AI Medical Chatbot (RAG + LLM)

- **FR-AI-1.1:** The system shall provide an AI medical chatbot accessible from both web and mobile interfaces.
- **FR-AI-1.2:** The chatbot shall support Arabic and English with automatic language detection.
- **FR-AI-1.3:** The chatbot shall answer general medical questions with a clear medical disclaimer and assist with platform navigation.
- **FR-AI-1.4:** The chatbot shall use RAG (Retrieval-Augmented Generation) over a curated medical knowledge base stored in FAISS/ChromaDB.
- **FR-AI-1.5:** The chatbot shall maintain context-aware conversations within a session and store persistent chat history per user (JSONB in PostgreSQL).
- **FR-AI-1.6:** The chatbot shall escalate to the Triage flow if symptoms are detected during conversation.
- **FR-AI-1.7:** The chatbot may support voice input (Speech-to-Text) as an optional mobile feature.

4.2 AI Medical Triage (ML Classifier)

- **FR-AI-2.1:** Before booking an appointment, patients shall optionally be guided through symptom collection (age, gender, symptoms, duration, severity, chronic conditions).
- **FR-AI-2.2:** A trained Scikit-learn model shall classify the case into: **simple**, **moderate**, or **urgent**.
- **FR-AI-2.3:** The system shall recommend a doctor or specialty based on the classification.
- **FR-AI-2.4:** The system shall flag emergency cases and strongly recommend calling emergency services.
- **FR-AI-2.5:** The system shall display a disclaimer that triage is informational, not a diagnosis.
- **FR-AI-2.6:** Triage results shall be logged for analytics and continuous model improvement.

4.3 Drug Interaction Detection

- **FR-AI-3.1:** The system shall analyze medication lists and detect dangerous interactions automatically.
- **FR-AI-3.2:** Interactions shall be classified by severity: **minor**, **moderate**, **major**, or **contraindicated**.
- **FR-AI-3.3:** The system shall alert doctors and patients before a prescription is finalized.
- **FR-AI-3.4:** The system shall provide an explanation of the interaction mechanism and reference sources.

4.4 Medical Report Summarization

- **FR-AI-4.1:** The system shall accept PDF report uploads and generate AI summaries using NLP (LLM).
- **FR-AI-4.2:** The summary shall extract: diagnosis, medications, and recommendations.
- **FR-AI-4.3:** The summary shall be stored alongside the original report and prominently displayed in the UI.

4.5 AI Analytics Dashboard

- **FR-AI-5.1:** The system shall provide appointment analytics (total, by status, by specialty).
- **FR-AI-5.2:** The system shall predict peak hours for bookings using historical time-series patterns.
- **FR-AI-5.3:** The system shall analyze doctor workload (appointments per doctor, utilization rate).
- **FR-AI-5.4:** The system shall predict specialty demand for upcoming periods.

- **FR-AI-5.5:** The system shall generate AI-generated natural-language insights for the admin dashboard.

5. External Interface Requirements

5.1 User Interfaces

- **Web Interface:** Responsive HTML5/CSS3/ES6+ application supporting 360px (mobile), 768px (tablet), and 1280px+ (desktop) breakpoints. Full RTL support for Arabic and LTR for English.
- **Mobile Interface:** Flutter 3.x cross-platform application with native performance, secure storage, and offline capability for cached data.
- **Admin Dashboard:** Web-based analytics and management interface with data visualization charts and export capabilities.

5.2 Hardware Interfaces

- **Client Devices:** Desktop computers, laptops, tablets, smartphones with camera and microphone for telemedicine.
- **Server Infrastructure:** Containerized deployment on Linux servers with horizontal scaling capability.
- **Network:** Standard TCP/IP with HTTPS; WebSocket for real-time; WebRTC for peer-to-peer media.

5.3 Software Interfaces

External System	Integration Purpose	Protocol
Google Maps API	Location search, nearby facilities, navigation	REST / JavaScript SDK
Firebase Cloud Messaging (FCM)	Mobile push notifications	HTTP / Admin SDK
SMTP Gateway	Email notifications (appointment reminders, password reset)	SMTP
OpenAI API / Ollama	LLM for chatbot, summarization, and insights	REST / HTTP
HuggingFace	Embedding models for RAG pipeline	REST / Local download
Google Translate API	Medical translation during international consultations (optional)	REST

5.4 Communications Interfaces

- **REST APIs:** HTTP/2 + JSON for synchronous client-server and service-to-service communication.
- **WebSocket:** Real-time bidirectional communication for chat, notifications, and WebRTC signaling.
- **WebRTC:** Peer-to-peer video/voice streaming for telemedicine consultations.

- **Apache Kafka:** Asynchronous event streaming for domain events (appointment lifecycle, prescription alerts, user registration).
 - **gRPC (optional):** High-performance service-to-service communication for internal microservices.
-

6. Non-Functional Requirements

6.1 Performance Requirements

- **NFR-1.1:** API response time shall be less than 2 seconds under normal load (95th percentile).
- **NFR-1.2:** Chat message delivery latency shall be less than 200 ms.
- **NFR-1.3:** Web application page load time shall be less than 2 seconds on a 4G connection.
- **NFR-1.4:** Mobile application cold start shall be less than 3 seconds on a mid-range device.
- **NFR-1.5:** The system shall support thousands of concurrent users with horizontal scaling.

6.2 Scalability Requirements

- **NFR-2.1:** The system shall adopt a microservices architecture enabling independent deployment and scaling of each service.
- **NFR-2.2:** Kafka topics shall be partitioned for parallel consumption by multiple service instances.
- **NFR-2.3:** Redis shall be used for caching frequently accessed data (doctor profiles, facility lists, session tokens).
- **NFR-2.4:** Services shall support stateless design to enable horizontal replication behind a load balancer.

6.3 Security Requirements

- **NFR-3.1:** All traffic shall be encrypted via HTTPS (TLS 1.2+).
- **NFR-3.2:** Passwords shall be hashed using bcrypt (cost factor 12).
- **NFR-3.3:** Sensitive medical data (diagnosis notes, attachments metadata) shall use AES-256 column-level encryption.
- **NFR-3.4:** RBAC shall be enforced at both the API Gateway and individual service levels.
- **NFR-3.5:** The system shall be protected against OWASP Top 10 vulnerabilities (SQL injection, XSS, CSRF).
- **NFR-3.6:** Rate limiting shall be applied at the gateway: 100 requests per minute per user/IP.
- **NFR-3.7:** All medical record access shall be audit-logged with user ID, timestamp, action, and IP address.

6.4 Reliability & Availability

- **NFR-4.1:** The system shall target 99.5% uptime for critical services (Auth, Appointment, Telemedicine).
- **NFR-4.2:** Critical services shall run with at least 2 instances for fault tolerance.
- **NFR-4.3:** Database backups shall run daily with point-in-time recovery capability.
- **NFR-4.4:** Kafka shall provide at-least-once delivery with dead-letter queues for failed message processing.
- **NFR-4.5:** Services shall implement circuit breakers (opossum) to prevent cascade failures.

6.5 Maintainability Requirements

- **NFR-5.1:** Each service shall expose `/health` and `/ready` endpoints for container orchestration.
- **NFR-5.2:** Each service shall publish OpenAPI 3.0 documentation via Swagger UI.
- **NFR-5.3:** The codebase shall follow a modular structure (monorepo or one repo per service) with low coupling and high cohesion.
- **NFR-5.4:** Services shall be independently deployable without coordinated releases.

6.6 Usability Requirements

- **NFR-6.1:** The web interface shall be fully responsive across mobile, tablet, and desktop viewports.
 - **NFR-6.2:** The system shall provide clear navigation, breadcrumbs, and helpful empty states.
 - **NFR-6.3:** The system shall support full RTL layout for Arabic and LTR for English with dynamic switching.
 - **NFR-6.4:** Error messages shall be user-friendly, bilingual, and actionable.
-

7. Architecture Overview

7.1 Microservices Architecture (15 Services)

MedOrbit adopts a microservices architecture split between two technology stacks to optimize for both I/O-bound core services and compute-bound AI services.

Node.js Services (11):

1. **API Gateway** – Single entry point, JWT validation, rate limiting, request routing, load balancing.
2. **Authentication Service** – Registration, login, OTP, JWT issuance, refresh tokens, password reset, RBAC.
3. **User Service** – User profiles, patient medical info, doctor professional info, preferences.
4. **Doctor Service** – Doctor catalog, specialty management, search & filter, ratings, verification workflow.
5. **Appointment Service** – Booking, rescheduling, cancellation, waiting list, smart suggestions, no-show handling.
6. **Medical Record Service** – EMR, SOAP records, medical history, chronic diseases, allergies, lab reports, uploads.
7. **Prescription Service** – Prescription CRUD, PDF generation, drug interaction invocation, medication reminders.
8. **Telemedicine Service** – Video/voice/chat orchestration, WebRTC signaling, file sharing, consultation notes.
9. **Notification Service** – Push (FCM), email (SMTP), in-app real-time (WebSocket), preference management.
10. **File Management Service** – Upload, store, retrieve, and secure medical documents and images with encryption.
11. **Location Service (GPS)** – Nearby hospitals/clinics/pharmacies search, Google Maps integration, Redis caching.

Python Services (4): 12. **AI Chatbot Service** – RAG + LLM medical chatbot (Arabic & English), context-aware conversations, persistent history. 13. **AI Triage Service** – Symptom collection, ML-based risk classification (simple/moderate/urgent), specialty recommendation. 14. **Drug Interaction Service** – Analyze medication lists, detect dangerous interactions, alert doctors. 15. **Analytics Service** – AI analytics dashboard, peak hours prediction, specialty demand forecasting, AI insights, report summarization.

7.2 Inter-Service Communication

Pattern	Used For	Technology
Synchronous	Client → Service, Service → Service queries	REST (HTTP/JSON)
Asynchronous	Domain events (appointment lifecycle, notifications, analytics)	Apache Kafka
Real-time	Chat, notifications, video signaling	WebSocket + WebRTC

7.3 Service-to-Service Authority Rules

- A service **never** queries another service's database directly.
- Cross-service data access **must** go through REST APIs or Kafka events.
- The **API Gateway** is the only public-facing entry point; internal service URLs are not exposed to clients.
- Each service owns its data and exposes it via well-defined APIs.

8. Technology Stack & Justification

8.1 Frontend – Web

Technology	Purpose	Justification
HTML5 + CSS3 + ES6+	Core web stack	Lightweight, no build-step dependency, direct browser execution, aligns with course requirements
Responsive CSS Grid/Flexbox	Layout system	Native browser support, no external library overhead
Fetch API	HTTP client	Native modern replacement for XMLHttpRequest
WebSocket API	Real-time communication	Native browser support for chat and notifications
Google Maps JavaScript API	Location visualization	Industry standard for mapping and navigation
Tailwind CSS (optional)	Utility-first styling	Rapid UI development with consistent design tokens
JSON Dictionary Files	i18n (AR/EN)	Simple, lightweight localization without heavy frameworks

8.2 Frontend – Mobile

Technology	Purpose	Justification
Flutter 3.x + Dart 3	Cross-platform mobile	Single codebase for Android/iOS, native performance, rich widget ecosystem
Riverpod	State management	Compile-safe, testable, and scalable state management for Flutter
Dio	HTTP client	Powerful HTTP client with interceptors, request cancellation, and global configuration
firebase_messaging	Push notifications	Official Firebase plugin for FCM integration
google_maps_flutter	Maps	Official Google Maps plugin for Flutter
flutter_webrtc	Video/voice calls	WebRTC implementation for Flutter enabling telemedicine
flutter_secure_storage	Sensitive data	Encrypted local storage for tokens and credentials

8.3 Backend – Core Services (Node.js)

Technology	Purpose	Justification
Node.js 20 LTS + JavaScript (ES Modules)	Runtime & language	Non-blocking I/O ideal for real-time healthcare; ES Modules for modern module system
Express.js / Fastify	Web framework	Express for ecosystem maturity; Fastify for performance and JSON schema validation
jsonwebtoken + bcrypt	Auth & security	Standard JWT implementation; bcrypt for secure password hashing
Swagger (OpenAPI 3.0)	API documentation	Auto-generated interactive docs for every service
kafkajs	Event streaming	Native Kafka client for Node.js with producer/consumer/ admin capabilities
Pino	Logging	High-performance structured JSON logging
Jest + Supertest	Testing	Industry-standard unit and integration testing for Node.js
node-pg-migrate	DB migrations	Lightweight PostgreSQL migration tool

8.4 Backend – AI Services (Python)

Technology	Purpose	Justification
Python 3.11	Runtime	De facto language for AI/ML with unmatched library ecosystem
FastAPI + Pydantic v2	Web framework	Async-native, auto-generated OpenAPI docs, strict data validation

Table 8 – continued

Technology	Purpose	Justification
LangChain	LLM orchestration	Simplifies RAG pipeline, prompt chaining, and agent development
OpenAI API / Llama (Ollama)	LLM provider	OpenAI for quality; Llama for free local deployment without API costs
FAISS / ChromaDB	Vector store	Efficient similarity search for RAG retrieval; ChromaDB for easier setup
Scikit-learn	ML classification	Mature, well-documented library for triage Random Forest/Gradient Boosting
Pytest	Testing	Python standard for testing with fixtures and parametrization
Alembic	DB migrations	SQLAlchemy-based migration tool for Python services

8.5 Data Layer & Infrastructure

Technology	Purpose	Justification
PostgreSQL 15+	Primary database	ACID compliance, JSONB support, GIN indexing, vector extensions
Redis	Cache & sessions	In-memory performance for session storage, GPS caching, and rate limiting
FAISS / ChromaDB	Vector database	Dedicated storage for RAG embeddings; decoupled from relational DB
Apache Kafka (KRaft)	Event backbone	Distributed, fault-tolerant, high-throughput event streaming without ZooKeeper
Docker + Docker Compose	Containerization	Consistent development and deployment environments
Nginx	Reverse proxy	Load balancing, SSL termination, static file serving
GitHub Actions	CI/CD	Automated testing and deployment on every push
Prometheus + Grafana	Monitoring	Metrics collection and visualization for service health

9. Project Phases & Deliverables

Phase 1: Foundation (Weeks 1–4)

- Docker Compose environment for all 15 services
- Authentication Service (JWT, OTP, RBAC, bcrypt)
- User Service & Doctor Service with PostgreSQL schemas
- Web frontend scaffold (HTML/CSS/JS) with RTL/LTR support
- Mobile Flutter scaffold with navigation and i18n

Phase 2: Core Healthcare (Weeks 5–8)

- Appointment Service (booking, cancellation, rescheduling, waiting list)
- Doctor availability management
- Medical Record Service (SOAP format, file uploads, version control)
- Prescription Service with PDF generation
- Notification Service (email + push)

Phase 3: Real-Time & AI (Weeks 9–12)

- Telemedicine Service (WebRTC signaling, WebSocket chat, file sharing)
- AI Chatbot Service (RAG pipeline, Arabic/English, persistent history)
- AI Triage Service (symptom collection, Scikit-learn classifier)
- Drug Interaction Service (rule-based + severity classification)
- Report Summarization (LLM extraction)

Phase 4: Analytics & Polish (Weeks 13–16)

- Analytics Service (dashboard, forecasting, AI insights)
- Location Service (GPS, Google Maps, Redis caching)
- Admin dashboard with data visualization
- Comprehensive testing (unit + integration 60% coverage)
- OpenAPI documentation for all services
- CI/CD pipeline (GitHub Actions)
- End-to-end demo preparation and documentation

10. Acceptance Criteria

The graduation project will be considered complete when the following criteria are satisfied:

#	Criterion
1	All 15 microservices are containerized and runnable via <code>docker-compose up</code>
2	Both web and mobile clients can register, login, and complete the full flow: book → consult → prescribe
3	All 5 AI features are functional: Chatbot (RAG), Triage, Drug Interaction, Summarization, Analytics
4	Each service has unit + integration tests with 60% code coverage
5	OpenAPI 3.0 documentation (Swagger UI) is published and accessible for every service
6	CI/CD pipeline builds, tests, and reports coverage on every GitHub push
7	Arabic (RTL) and English (LTR) are fully supported across web and mobile interfaces
8	Kafka events are observable; JSONB chat history is persisted and retrievable; RAG pipeline works end-to-end
9	GPS feature returns real nearby facilities; JWT/RBAC security is verified and penetration-tested

Table 10 – continued

#	Criterion
10	End-to-end demo is demonstrated at final defense with architecture diagrams and README documentation
11	Database migrations run cleanly from scratch for all services
12	Audit logging is verified for all medical record access events

MedOrbit – Smart Healthcare Platform

Software Requirements Specification v2.2

Prepared for Advanced Software Engineering – Graduation Project